Part 1: Introduction to R

307312 Applied Statistics — Masters Level

R performs calculations and statistical analysis. **RStudio** is an interface for writing code, inspecting data, and viewing plots. **Quarto** combines explanations, executable code, and results in one document.

By the end of this notebook, you should be able to import a CSV, inspect its variables, prepare and summarize data with **dplyr**, and create clear charts with **ggplot2**.

1.1 Getting started

Install and open the course project

1. Install R and RStudio on your computer.
2. Keep the full course folder structure: notebooks in `lessons/`, with `data/` and `assets/` in the course root.
3. Open `Applied-Statistics.Rproj` in RStudio. This makes the course folder the working directory.
4. Open this `.qmd` file. Run individual code cells to explore, or choose **Render** to rebuild the complete lesson. Rendering also requires Quarto.

The **Console** runs commands immediately. The **source editor** saves your work. The **Environment** lists saved objects, and **Plots** displays charts. Keep useful commands in the notebook rather than only in Console history.

Install packages once; load them each session

A package adds functions to R. Install the two packages below once on your computer. This cell is provided for manual setup and does not run during rendering.

```r
# install.packages() downloads packages and their dependencies; run this once with an internet connection.
# c() combines the two package names into a character vector.
install.packages(c("dplyr", "ggplot2"))
```

Load the packages whenever you start a new R session or render the notebook.

```r
# library() loads an installed package so its functions are available in the current session.
# dplyr prepares tables; ggplot2 builds plots from data and visual mappings.
library(dplyr)
library(ggplot2)
```

1.2 Essential R concepts

Objects, assignment, and data types

Use `<-` to save a value under a name. R distinguishes uppercase and lowercase: `height_cm` and `Height_cm` are different names. Text goes inside quotes; logical values are `TRUE` or `FALSE`. A line beginning with `#` is a comment.

```r
# <- assigns values to names; arithmetic uses the saved values on the right-hand side.
# These practice values demonstrate numeric, character, and logical objects, rather than a new dataset.
height_cm <- 172
height_m <- height_cm / 100
course_name <- "Applied Statistics"
is_introductory <- TRUE
height_m
```

```
[1] 1.72
```

Vectors and selecting values

A **vector** holds several values of one basic type. Most arithmetic and comparisons work on every element without a loop. R starts counting positions at **1**.

```r
# c() combines these practice scores into a vector; [] selects values by position or a logical
        condition.
# > compares each score with 75; only entries satisfying that condition are retained.
scores <- c(70, 80, 90)
scores + 5
```

```
[1] 75 85 95
```

```r
scores[1]
```

```
[1] 70
```

```r
scores[scores > 75]
```

```
[1] 80 90
```

`==` tests equality, `!=` means not equal, `&` means AND, and `|` means OR. `%in%` tests membership in a set. Use `<-` for assignment and `==` when comparing values.

Basic summaries and missing values

```r
# mean() and median() summarize center; sd() uses the sample denominator n-1 to measure spread.
# c() collects named results; scores are practice values, not observations added to the teaching CSV.
c(mean = mean(scores), median = median(scores), sample_sd = sd(scores))
```

```
     mean    median sample_sd 
       80        80        10 
```

`NA` represents a missing value. It is different from zero and from an empty string. Do not replace missing values with zero automatically.

```r
# is.na() marks missing entries; mean() returns NA by default when any input is missing.
# na.rm=TRUE omits missing entries, so this mean uses only the two recorded scores, 70 and 90.
scores_with_missing <- c(70, NA, 90)
is.na(scores_with_missing)
```

```
[1] FALSE  TRUE FALSE
```

```r
mean(scores_with_missing)
```

```
[1] NA
```

```r
mean(scores_with_missing, na.rm = TRUE)
```

```
[1] 80
```

Reading function calls and getting help

A function call has a name followed by arguments in parentheses: `mean(scores, na.rm = TRUE)`. Here `scores` is the input and `na.rm` changes how missing values are handled. Naming consequential arguments makes code easier to understand.

To open R's help, type `?mean` or `?read.csv` in the Console. Read error messages from the first failing line; check spelling, package loading, column names, and file paths before changing the data.

A small reusable function

A function saves a calculation that you want to reuse. R returns the final expression when there is no explicit `return()`.

```r
# function() defines a reusable calculation; x is the input supplied when the function is called.
# Dividing by 100 converts centimetres to metres and works on a whole numeric vector.
cm_to_m <- function(x) {
  x / 100
}
cm_to_m(c(160, 172, 168))
```

```
[1] 1.60 1.72 1.68
```

1.3 Importing and checking a dataset

The course example

We will use the same teaching CSV as Part 2. Each row represents one student record in the original 307102 teaching example. The selected columns are `gender`, `age`, `height`, `weight`, `country`, and `major`. Age is in **years**, height in **cm**, and weight in **kg**.

The original collection provenance is not established; do not describe these as verified records of actual university students. The CSV retains the source values and omits names and identifiers. See the data guide.

Read a CSV into a data frame

A **data frame** is a table: rows are observations, and columns are variables. CSV files are plain-text tables with values separated by commas.

```
# read.csv() reads column names from the first row; stringsAsFactors=FALSE keeps text columns as text.
# The path is relative to the course project folder, not an absolute path on the instructor's computer.
# Find the shared data folder from either the course root or this notebook folder.
data_dir <- if (dir.exists("data")) "data" else file.path("..", "data")
students <- read.csv(file.path(data_dir, "descriptive-statistics/students.csv"), stringsAsFactors =
        FALSE)
```

Inspect before analyzing

```
# head() previews six rows; dim() reports row and column counts, and names() lists the column names.
# str() shows column types and example values, helping detect numbers imported as text.
head(students)
```

```
  gender age height weight country                  major
1 Female  19    160   64.8  Turkey      Political Science
2 Female  19    172   73.0   Other        Social Sciences
3 Female  22    168   70.6      UK        Social Sciences
4   Male  19    183   79.7   Other Environmental Sciences
5 Female  21    175   71.4   Italy Environmental Sciences
6   Male  19    189   85.8   Italy      Political Science
```

```
dim(students)
```

```
[1] 8239    6
```

```
names(students)
```

```
[1] "gender"  "age"     "height"  "weight"  "country" "major"
```

```
str(students)
```

```
'data.frame':	8239 obs. of  6 variables:
 $ gender : chr  "Female" "Female" "Female" "Male" ...
$ age    : int  19 19 22 19 21 19 21 21 18 18 ...
$ height : int  160 172 168 183 175 189 156 167 195 165 ...
$ weight : num  64.8 73 70.6 79.7 71.4 85.8 65.9 65.7 94.4 66 ...
$ country: chr  "Turkey" "Other" "UK" "Other" ...
$ major  : chr  "Political Science" "Social Sciences" "Social Sciences" "Environmental Sciences" ...
```

```r
# is.na() identifies missing cells and colSums() counts them separately for each column.
# summary() gives an overview appropriate to each column's type; it does not verify data provenance.
colSums(is.na(students))
```

```
gender     age  height  weight country   major 
     0       0       0       0       0       0 
```

```r
summary(students[c("age", "height", "weight")])
```

```
     age             height          weight     
Min.   :18.00   Min.   :135.0   Min.   : 51.4  
1st Qu.:20.00   1st Qu.:163.0   1st Qu.: 66.5  
Median :21.00   Median :171.0   Median : 71.8  
Mean   :22.54   Mean   :171.4   Mean   : 73.0  
3rd Qu.:23.00   3rd Qu.:180.0   3rd Qu.: 78.5  
Max.   :64.00   Max.   :206.0   Max.   :116.0  
```

Use `$` to select a column, such as `students$height`. Use `students[1:5, ]` to select the first five rows; the blank position after the comma keeps every column. Use `students[c("age", "height")]` to select named columns.

Categories and factors

The `|>` pipe passes a table into the next operation. In the example below, `mutate()` changes a column; Section 1.4 develops these operations further.

A **factor** tells R that values represent categories. Category labels should not be averaged as though they were numerical measurements. The source spells Russia as `Russa`; retain that label when filtering the unchanged data.

```r
# factor() records gender as a categorical variable; mutate() creates an updated table without editing
        the CSV.
# This assignment changes the in-memory students object; the original file on disk stays unchanged.
students <- students |>
  mutate(gender = factor(gender))
```

1.4 Preparing and summarizing data with dplyr

Read a pipe from left to right

The native pipe `|>` sends the result on its left into the next function. Read a chain as a sequence of steps: **start with this table, then perform this operation**. It requires R 4.1 or later.

Function	Purpose
`select()`	Choose columns

Function	Purpose
`filter()`	Keep rows meeting conditions
`arrange()`	Sort rows
`mutate()`	Create or change columns
`count()`	Count rows by category
`group_by()` and `summarise()`	Calculate summaries within groups

Select, filter, and sort

```
# select() chooses columns; filter() keeps rows satisfying both conditions separated by a comma.
# arrange(desc()) sorts height from largest to smallest; head(6) limits the displayed output to six
        rows.
students |>
  select(gender, age, height, country) |>
  filter(age >= 20, country == "Turkey") |>
  arrange(desc(height)) |>
  head(6)
```

```
  gender age height country
1   Male  23    206  Turkey
2   Male  21    198  Turkey
3   Male  23    196  Turkey
4   Male  21    195  Turkey
5   Male  22    194  Turkey
6   Male  20    193  Turkey
```

A comparison involving `NA` does not pass `filter()`. Investigate missing values before interpreting the rows that remain.

Create a derived variable

```
# mutate() adds height_m while keeping existing columns; cm_to_m() is the conversion function defined
        above.
# select() and head() display the original and converted heights together, with their units explicit in
        the names.
students_prepared <- students |>
  mutate(height_m = cm_to_m(height))
students_prepared |>
  select(height, height_m) |>
  head(6)
```

```
  height height_m
1    160     1.60
2    172     1.72
3    168     1.68
4    183     1.83
5    175     1.75
6    189     1.89
```

Count categories and calculate percentages

```r
# count() returns one row per country and sorts by frequency when sort=TRUE; n is its default count column.
# mutate() adds percentages using sum(n): the denominator is all records counted across the country categories.
country_counts <- students |>
  count(country, sort = TRUE) |>
  mutate(percent = 100 * n / sum(n))
country_counts
```

```
  country    n   percent
1   Italy 2797 33.948295
2   Other 2688 32.625319
3      UK 1839 22.320670
4   Russa  585  7.100376
5  Turkey  330  4.005340
```

Summarize within groups

```r
# group_by() defines gender groups; summarise() calculates one row of statistics per group.
# n() counts rows, mean() and sd() summarize height in cm, and .groups="drop" removes grouping afterward.
height_summary <- students |>
  group_by(gender) |>
  summarise(n = n(), mean_height_cm = mean(height), sd_height_cm = sd(height),
            .groups = "drop")
height_summary
```

```
# A tibble: 2 × 4
  gender     n mean_height_cm sd_height_cm
  <fct>  <int>          <dbl>        <dbl>
1 Female  4110           164.         7.92
2 Male    4129           179.         7.99
```

If a variable has missing observations, check how many values contribute to each statistic. Adding `na.rm = TRUE` changes the denominator; `n()` still counts all rows, including rows missing that variable.

Save a derived result

Keep input files unchanged. This optional cell writes a new result to `_output/`; run it manually when you want to save the summary.

```r
# dir.create() prepares an output folder; showWarnings=FALSE suppresses a message if it already exists.
# write.csv(row.names=FALSE) saves the summary without adding a column of row numbers.
dir.create("_output", showWarnings = FALSE)
write.csv(height_summary, "_output/height-summary-by-gender.csv", row.names = FALSE)
```

1.5 Visualizing data with ggplot2

Data, mappings, and layers

A ggplot starts with a table and a mapping, then adds a geometry with `+`:

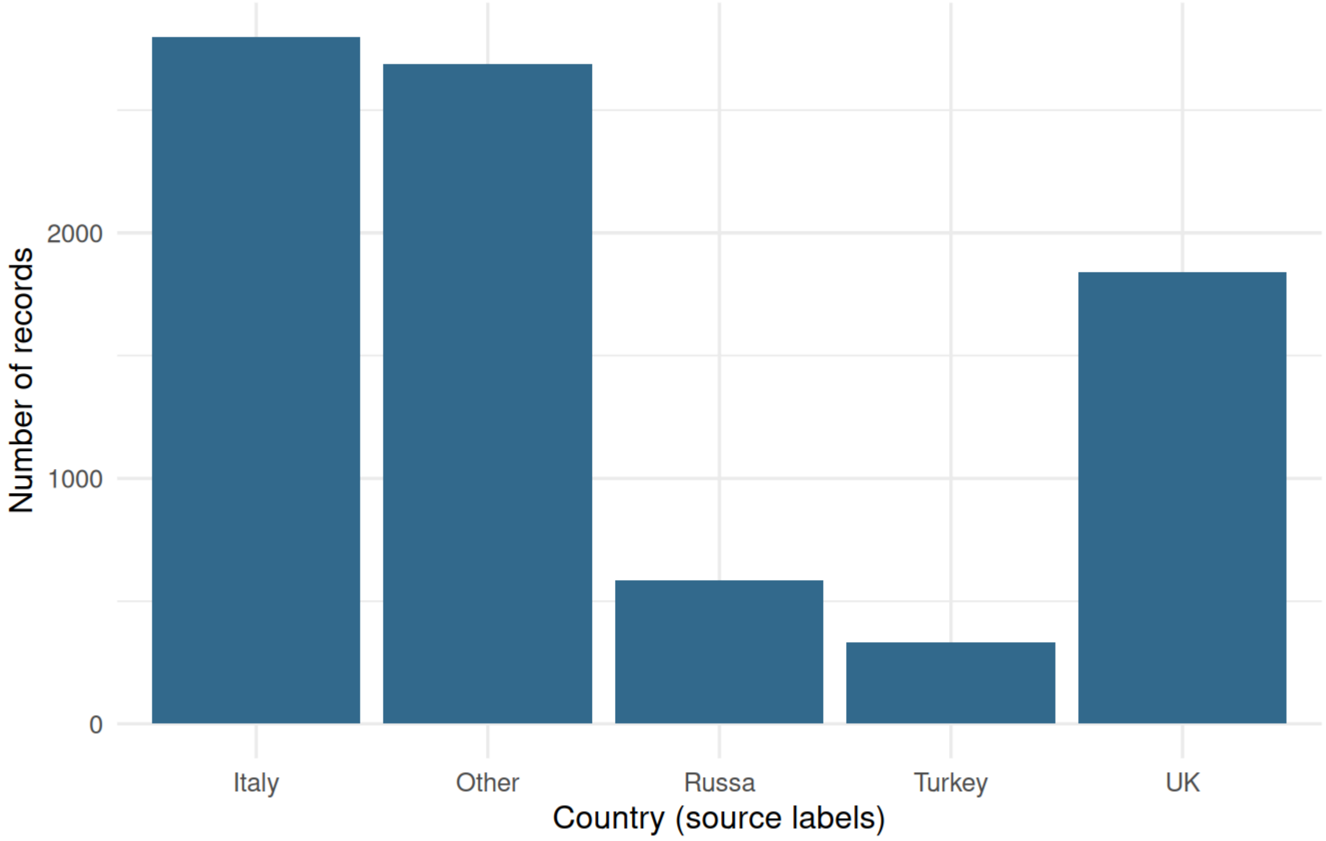
```
ggplot(data, aes(x = variable_x, y = variable_y)) +
  geom_point()
```

This is a syntax template. The examples below supply actual column names. `aes()` maps variables to visual properties. Put a fixed color **outside** `aes()`; put a variable **inside** `aes()` when color should distinguish groups.

```
# theme_set() changes the default ggplot theme for the current R session, including later plots in this
        notebook.
# theme_minimal(base_size=12) uses a simple background and sets the base text size to 12 points.
theme_set(theme_minimal(base_size = 12))
```

Bar chart: count categories

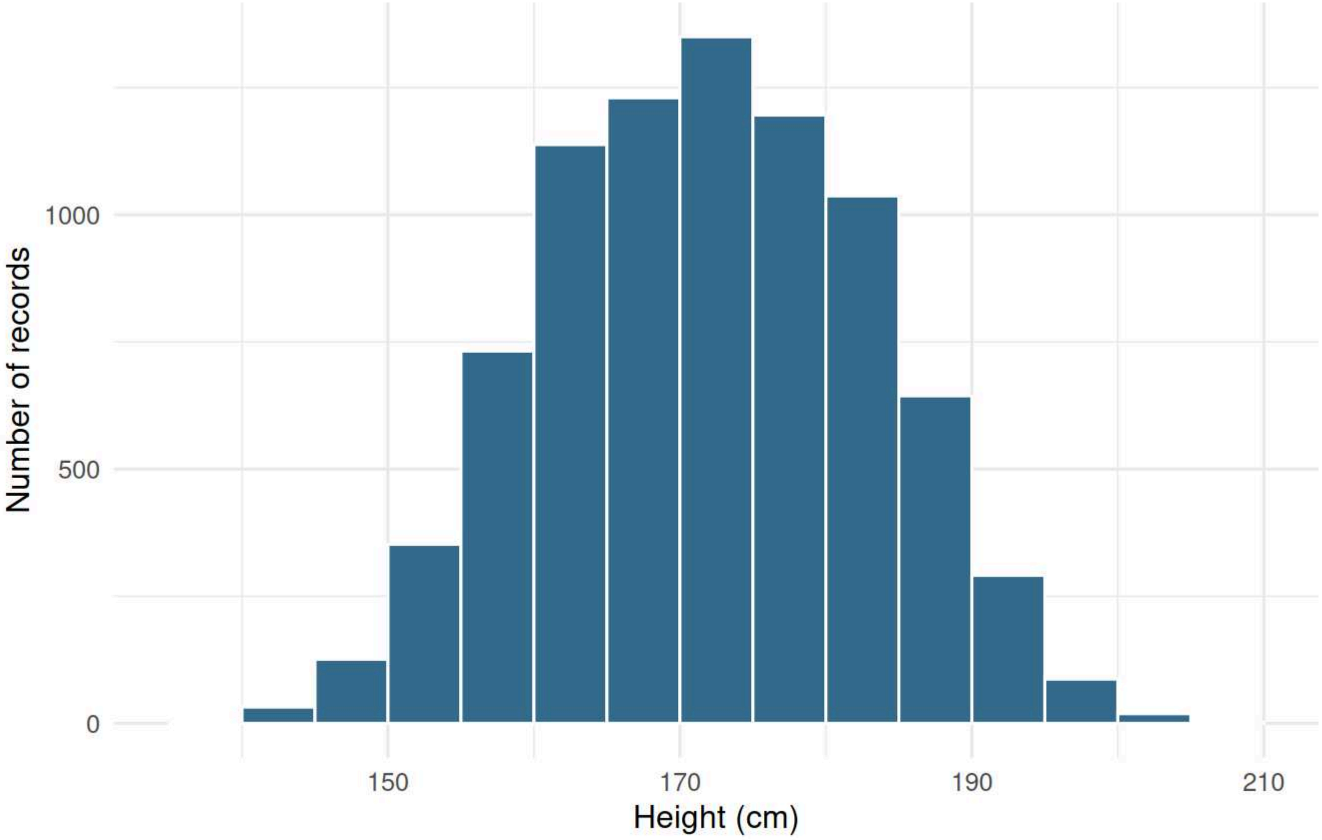
```
# ggplot() supplies the table; aes(x=country) maps the country column to the horizontal axis.
# geom_bar() counts rows by default; fill is a fixed color, and labs() gives axes meaningful labels.
ggplot(students, aes(x = country)) +
  geom_bar(fill = "#356B8C") +
  labs(x = "Country (source labels)", y = "Number of records")
```



`geom_bar()` counts raw rows. If you already have a table of calculated values, use `geom_col()` instead; it plots the supplied heights rather than counting again.

Histogram: display a numerical distribution

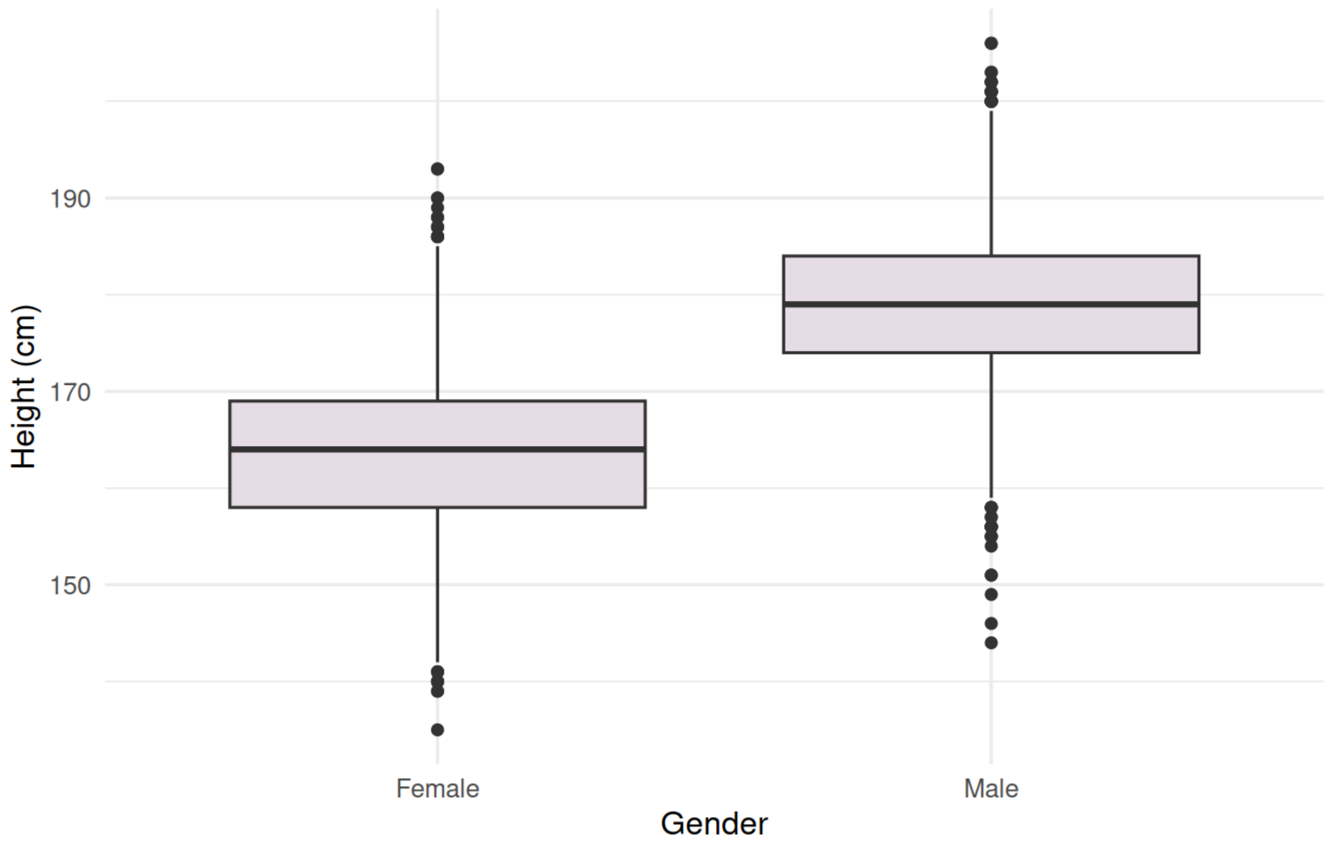
```
# geom_histogram() counts heights within intervals; binwidth=5 means bins span 5 cm.
# boundary=0 anchors bin edges at multiples of 5 cm; closed="left" includes the lower edge of each bin.
ggplot(students, aes(x = height)) +
  geom_histogram(binwidth = 5, boundary = 0, closed = "left", fill = "#356B8C", color = "white") +
  labs(x = "Height (cm)", y = "Number of records")
```



Changing the bin width can change the apparent shape. Parts 2 and 3 develop the statistical interpretation; here the aim is to construct and label the plot correctly.

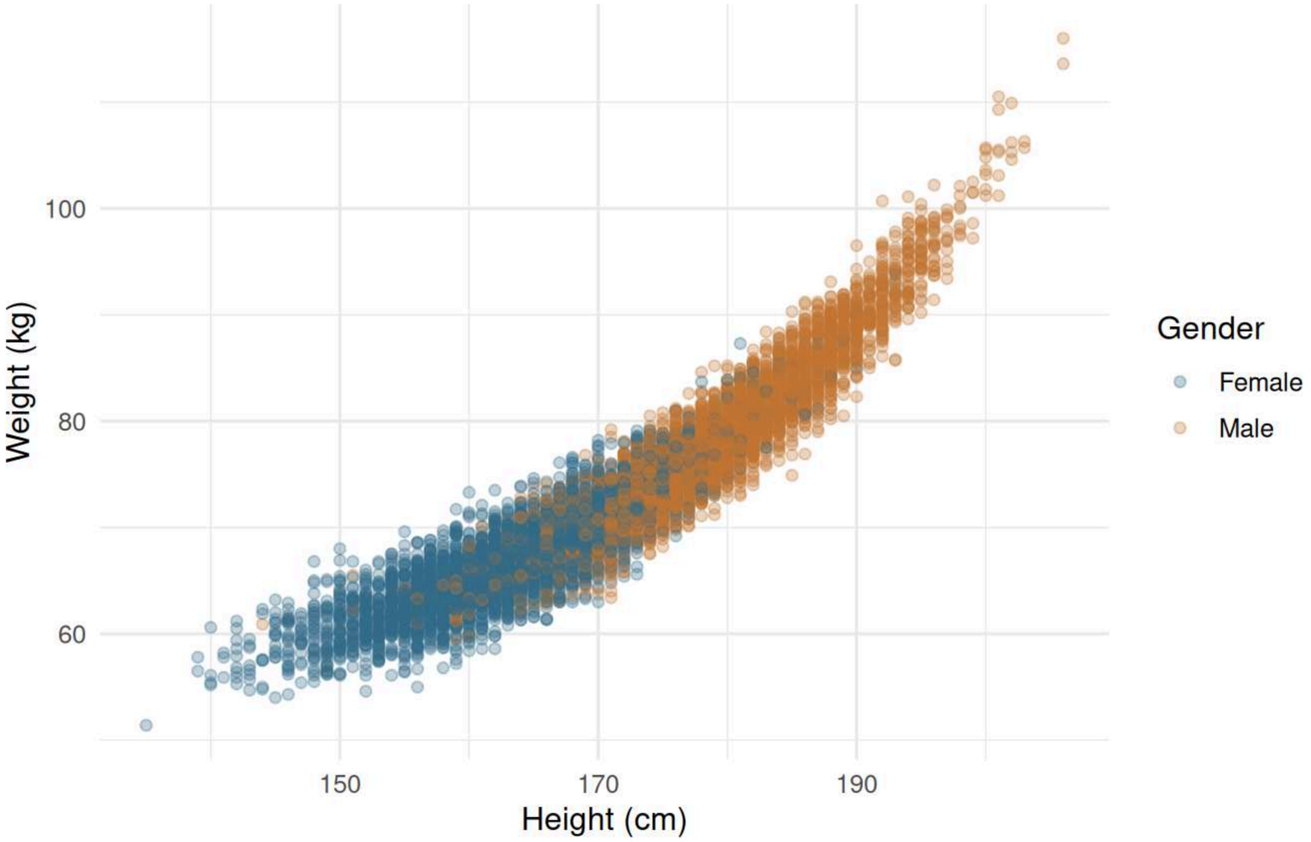
Boxplot: compare groups

```
# geom_boxplot() displays each gender group's distribution of height; its whiskers extend to
        observations within 1.5 IQR of the hinges.
# Values beyond the whiskers are plotted separately as potential outliers, not automatically as errors.
ggplot(students, aes(x = gender, y = height)) +
  geom_boxplot(fill = "#E9DDE7") +
  labs(x = "Gender", y = "Height (cm)")
```



Scatterplot: examine two numerical variables

```r
# aes(color=gender) maps categories to point colors; geom_point(alpha=0.3) makes overlapping records
        more visible.
# scale_color_manual() assigns fixed analytical colors to named groups; labs() labels axes and the
        legend.
ggplot(students, aes(x = height, y = weight, color = gender)) +
  geom_point(alpha = 0.3) +
  scale_color_manual(values = c(Female = "#356B8C", Male = "#C47735")) +
  labs(x = "Height (cm)", y = "Weight (kg)", color = "Gender")
```



A scatterplot shows association, not proof that changing one variable causes the other to change.

1.6 A short practice

Using the supplied CSV and the examples above:

1. Select students from Turkey or `Russa` using `%in%`, then display their age, height, and country.
2. Count records by major. Identify the major with the largest count.
3. Calculate the mean weight in kg for each gender and report the group sizes.
4. Make a histogram of age with a bin width of one year. Label both axes.
5. Make a scatterplot of height and weight using a fixed point color. Explain how this differs from mapping gender to color.

Keep your code and results together. Check variable types, missing values, units, and the meaning of each row before interpreting a result.

1.7 What to remember

- Use meaningful object names and save reusable code in the notebook.
- Inspect a CSV before analyzing it; a successful import does not establish data quality.
- Use dplyr to select, filter, transform, count, and summarize.
- Build ggplot2 charts from data, mappings, and geometry, with explicit units and labels.

Continue to Part 2: Descriptive Statistics · Course home